# Simulation Object Standard - (SOS)

**OOF™ Origin Open Foundation™**

Independent Methodological Authority

**Architecture Ecosystem:** Structured Reality Standards™
**Architecture Family:** SIMULOS® — Simulation Governance Architecture
**Parent Standard:** Simulation Object Standard (SOS)
**Operational Layer:** Simulation Object Governance Layer
**Category:** Governance & Enforcement
**Subcategory:** Simulation Governance
**Type:** Parent Standard
**Governed Space:** Simulation Object
**Version:** 1.0
**Status:** Canonical · Open Standard
**Effective Date:** 15 August 2026
**Compatibility:** OOF® Methodology OS™ · GOA™ · ORA™ · VALIDOS® · INTEGROS® · ADIT® · AGA™ · AIG® · CLIA® · MGIA™ · ASGA™ · RIS™
**AI-Readable:** Yes
**Authority:** OOF®
**Protection:** MIP® — Methodological Intellectual Property
**Canonical Language:** English (UCL™)

## Governed Space

## Simulation Object

---

## Canonical Definition

Simulation Object Standard (SOS) defines the governance conditions under which the exact object, system, process, behavior, environment, event, interaction, infrastructure, population, mission, organization, or other reality target intended to be represented through simulation is formally identified, bounded, distinguished, versioned, configured, and documented as a governed Simulation Object.

SOS establishes the foundational object layer of SIMULOS® by ensuring that simulation governance begins with an explicitly defined and traceable simulation target rather than an ambiguous, unstable, retrospectively reconstructed, or silently changing object.

---

## Core Question

What exactly is being simulated, and how must the Simulation Object be identified, bounded, versioned, configured, and documented so that every downstream simulation activity remains connected to the same governed object?

---

## Purpose

The purpose of SOS is to establish the object foundation upon which the complete SIMULOS® lifecycle depends.

Simulation cannot be meaningfully governed if the object it claims to represent is unclear.

Before SIMULOS® can govern: Simulation Purpose & Scope, Reality Representation, Simulation Assumptions, Simulation Model & Environment, Simulation Inputs & Data, Scenario Design, Simulation Execution, Simulation Output Governance, Simulation-to-Reality Correlation, simulation-derived evidence, or lifecycle resimulation,

the architecture must first establish:

What exactly is being simulated?

---

## Operational Role

SOS is the first Parent Standard of SIMULOS® — Simulation Governance Architecture.

It transforms an intended simulation target into a formally governable Simulation Object.

Its role is to ensure that downstream simulation activities remain traceably connected to:

the same object, the correct version, the relevant configuration, the correct boundaries, and the applicable object state.

---

## Governance Flow

Candidate Simulation Target → Simulation Object Identification → Object Boundary → Object Structure → Object Version → Object Configuration → Object State → Object Documentation → Governed Simulation Object

The governed Simulation Object becomes the input to:

## Simulation Purpose & Scope Governance

## Simulation Object

A Simulation Object is the formally governed reality target whose selected characteristics, states, behaviors, relationships, interactions, processes, or conditions are intended to be represented through simulation.

A Simulation Object may be: physical, digital, biological, behavioral, organizational, environmental, infrastructural, economic, cognitive, autonomous, hybrid, process-based, event-based, population-based, mission-based, system-based, system-of-systems based, conceptual, or future-state based.

The Simulation Object defines what the simulation claims to represent.
Simulation Object ≠ Simulation Representation

SIMULOS® distinguishes between:

## Simulation Object

and:

## Simulation Representation

The Simulation Object is the governed subject being simulated.

The Simulation Representation is the constructed representation through which selected characteristics of that object are later reproduced or examined.

Therefore:

## Object ≠ Representation Simulation Object ≠ Simulation Model

The Simulation Object must also remain distinct from the simulation model.

For example:

## Operational Robot

may be the Simulation Object.

A mathematical, digital, behavioral, or hybrid model of the robot is not the object itself.

Therefore:

Simulation Object ≠ Simulation Model Simulation Object ≠ Simulation Environment

The environment surrounding an object may materially affect simulation, but it does not automatically become part of the Simulation Object.

A governed distinction must exist between:

Object

Environment

and:

## Object–Environment Interaction

unless the simulation intentionally defines them as one composite object.

## Simulation Object Identification

Every Simulation Object must be sufficiently identifiable to distinguish it from materially different objects.

Relevant identity characteristics may include: object name, object identifier, object class, system identity, product identity, mission identity, process identity, population identity, object version, configuration, state, component structure, relevant dependencies, and applicable temporal condition.

The required identification depth depends upon what differences could materially affect downstream simulation.

## Object Distinction

A generic category is insufficient where materially different members of the category may behave differently in simulation.

For example:

## Autonomous Vehicle

may be insufficient where the simulation outcome materially depends upon:

Vehicle Platform + Hardware Version + Software Version + Sensor Configuration + Autonomy Configuration

SIMULOS® therefore requires Simulation Object identification at the level necessary to preserve simulation meaning.

## Simulation Object Boundary

Every Simulation Object must have a defined boundary.

The boundary establishes:

what belongs to the Simulation Object

and:

what remains outside it.

The boundary may be: physical, digital, logical, functional, organizational, behavioral, temporal, spatial, systemic, or hybrid.

## Object Inclusion

Material components, actors, processes, functions, relationships, and dependencies included in the Simulation Object must remain identifiable.

Inclusion should be deliberate rather than inferred after simulation.

## Object Exclusion

Material exclusions must remain visible.

A simulation of part of a system must not silently become represented as simulation of the whole system.

SIMULOS® therefore establishes:

Object exclusion must remain visible wherever it materially limits simulation meaning.

## Simulation Object Structure

Complex Simulation Objects may contain multiple governed levels.

These may include:

## Composite Object

## System Object

## Subsystem Object

## Component Object

## Dependent Object

and:

## External Object

Where multiple levels exist, findings from one level must not automatically be generalized to another.

## Composite and System-of-Systems Objects

Multiple interacting systems may collectively form a Simulation Object.

In such cases SOS preserves:

## Composite Object Identity

while maintaining visibility of the identities and boundaries of materially relevant subordinate objects.

This allows SIMULOS® to govern: vehicles and infrastructure, spacecraft and ground systems, robot fleets, AI agents and tools, industrial production systems, energy networks, or other system-of-systems simulations

without collapsing all components into one indistinguishable object.

## Object Dependencies

A Simulation Object may depend upon systems or conditions outside its boundary.

Dependencies may include: external infrastructure, software services, communications, human operators, data services, energy, environmental conditions, external control systems, organizational dependencies, or other external objects.

Dependency does not automatically establish object inclusion.

The relationship must remain explicit.

## Simulation Object Version

Where the Simulation Object can materially change over time, the applicable version must be identifiable.

Version change may arise from: hardware modification, software update, component replacement, architecture revision, process revision, capability change, mission revision, organizational change, or another material modification.

## Version Binding

A governed simulation must remain traceable to the object version it claims to represent.

Conceptually:

## Simulation → Simulation Object → Object Version

A simulation performed against one materially distinct version may not automatically be represented as applying to another.

## Simulation Object Configuration

Version alone may not sufficiently define the Simulation Object.

The same version may operate under multiple configurations.

Configuration may include: enabled capabilities, disabled capabilities, software settings, sensor configuration, actuator configuration, payload, autonomy level, permissions, operating mode, control configuration, safety configuration, network configuration, dependency configuration, or mission configuration.

Where these materially affect simulated behavior, they form part of governed Simulation Object definition.

## Configuration Binding

Simulation results must remain connected to the configuration actually represented.

Therefore:

## Simulation of Configuration A ≠ Simulation of Configuration B

unless a governed basis demonstrates sufficient applicability between them.

## Simulation Object State

A Simulation Object may change materially without receiving a formal new version.

Relevant object state may include: operational state, degradation state, maintenance state, failure state, learning state, load state, damage state, resource state, or another material condition.

Where object state affects simulation meaning, the state must remain identifiable.

## Temporal Object Condition

Some Simulation Objects must be bound to a point in time or period.

Therefore:

## Object at T1

may not be equivalent to:

## Object at T2

where material change occurred between those states.

SOS governs the temporal identity necessary to preserve this distinction.

## Dynamic Simulation Objects

Dynamic objects may continuously evolve.

Examples include: adaptive AI systems, autonomous agents, organizations, populations, biological systems, economic systems, infrastructure networks, and ecosystems.

For dynamic objects, SOS requires a sufficiently bounded:

## Object State

or:

## Object-State Range

so that downstream simulation does not rely on an undefined moving target.

## Object Change

Material Simulation Object change may affect downstream simulation governance.

Relevant changes include: identity change, boundary change, version change, configuration change, component change, dependency change, capability change, state change, or structural change.

## Object Change Propagation

SIMULOS® establishes:

Material change to the Simulation Object must propagate to every downstream simulation space materially dependent upon that object definition.

SOS identifies the object-level change.

Subsequent SIMULOS® Parent Standards determine its effect on: purpose, representation, assumptions, models, inputs, scenarios, execution, outputs, correlation, evidence, and lifecycle relevance.

## Object Equivalence

Two objects must not be treated as simulation-equivalent solely because they share: a name, product family, general function, manufacturer, architecture, or superficial similarity.

Object equivalence must be sufficient relative to the simulation question.

## Object Substitution

Where one object is used as a proxy or substitute for another, the substitution must remain explicit.

The record must distinguish:

## Target Simulation Object

from:

## Proxy or Substitute Object

A proxy must not silently become the object the simulation claims to represent.

## No Object Inflation Rule

A simulation of:

one component

must not silently become:

simulation of the complete system.

A simulation of:

one configuration

must not silently become:

simulation of all configurations.

A simulation of:

one version

must not silently become:

simulation of the full product family.

## No Silent Object Change Rule

A materially changed Simulation Object must not continue through the simulation lifecycle under an unchanged object definition where doing so would conceal a material difference.

## No Retrospective Object Redefinition Rule

The Simulation Object must not be materially redefined after simulation results become known merely to increase apparent applicability of those results.

Material redefinition requires explicit governance and downstream reassessment.

## Simulation Object Documentation

The Simulation Object must remain sufficiently documented so that a qualified reviewer can determine:

what was actually intended to be simulated.

Documentation should preserve, where materially relevant: identity, boundary, inclusions, exclusions, version, configuration, state, component structure, dependencies, temporal applicability, and object changes.

## Simulation Object Traceability

The architecture should support reconstruction of:

## Simulation → Simulation Object → Version → Configuration → State

to the level required by the simulation purpose.

---

## Operational Space

SOS governs: Simulation Object identification, object distinction, object boundaries, inclusions and exclusions, composite and nested object structures, dependencies and interfaces, object versions, object configurations, object states, temporal applicability, proxy and substitute objects, object change, object equivalence, object documentation, and object traceability.

SOS does not govern how the object is later represented inside the simulation.

---

## Governance Boundary

SOS begins when:

a candidate target is proposed for simulation.

SOS ends when:

the Simulation Object has been sufficiently identified, bounded, distinguished, versioned, configured, state-defined, and documented to support governed Simulation Purpose & Scope.

---

## Governance Scope

SOS governs only Simulation Object governance.

It does not govern: why the simulation is performed, complete simulation scope, representation fidelity, simulation assumptions, model construction, simulation environment construction, input-data suitability, scenario design, execution controls, output interpretation, simulation-to-reality correlation, general validation, general integrity, audit governance, accountability governance, or operational deployment authorization.

These responsibilities belong to subsequent SIMULOS® Parent Standards or adjacent OOF® architectures.

---

## Minimum Governance Requirements

A conforming application of SOS should establish:

**1. an identifiable Simulation Object, 2. sufficient object distinction, 3. an explicit Simulation Object boundary, 4. material inclusions and exclusions, 5. material component and object relationships, 6. material external dependencies, 7. applicable object version, 8. applicable object configuration, 9. relevant object state, 10. relevant temporal applicability, 11. visibility of proxy or substitution relationships, 12. material object-change traceability, 13. prevention of silent object inflation, 14. prevention of retrospective object redefinition, 15. and a persistent Simulation Object Record.**

---

## Governance Outputs

SOS may produce: Simulation Object Identifier, Simulation Object Definition, Simulation Object Boundary Record, Object Inclusion Record, Object Exclusion Record, Simulation Object Structure Map, Composite Object Record, Object Dependency Record, Object Interface Record, Simulation Object Version Record, Simulation Object Configuration Record, Simulation Object State Record, Temporal Object Record, Proxy Object Declaration, Object Equivalence Record, Object Change Record, Simulation Object Traceability Record, Simulation Object Status, and Simulation Object Governance Record.

---

## Operational Applications

Simulation Object Standard may be applied across: artificial intelligence, autonomous systems, robotics, aerospace, space missions, aviation, autonomous vehicles, industrial systems, manufacturing, digital twins, critical infrastructure, healthcare, financial systems, scientific simulation, environmental modeling, infrastructure planning, mission simulation, organizational simulation, population simulation, policy simulation, system-of-systems environments, and future operational ecosystems.

---

## Relationship to Other OOF® Architectures

Simulation Object Standard interoperates with: GOA™ ORA™ OBIDENITY® VALIDOS® INTEGROS® ADIT® AGA™ AIG® CLIA® MGIA™ ASGA™ RIS™

by establishing the simulation-specific object foundation consumed by downstream simulation governance.

ORA™ may define or observe the operational reality from which a Simulation Object is derived.

OBIDENITY® may govern broader identity, origin, ownership, provenance, and authority relationships.

VALIDOS® may later treat the simulation, simulation output, or simulation-derived claim as a Validation Object.

INTEGROS® governs general integrity conditions affecting objects and records.

ADIT® may preserve evidential and audit traceability surrounding simulation activity.

AGA™ governs accountability for decisions involving the Simulation Object.

Specialized AI, cognition, memory, and autonomous-system architectures may govern characteristics of the actual system being simulated.

SOS does not replace those architectures.

It governs specifically:

what the simulation claims to represent.

---

## Architectural Position

Simulation Object Standard is the first Parent Standard of SIMULOS® — Simulation Governance Architecture.

It establishes the first transition in the simulation governance lifecycle:

## Candidate Simulation Target → Governed Simulation Object

The next architectural question is:

Why is this object being simulated, what must the simulation cover, and what remains outside its intended scope?

This transition leads directly into:

## Simulation Purpose & Scope Governance

---

## Foundational Principle

A simulation cannot legitimately claim to represent an object more precisely than that object has been identified, bounded, versioned, configured, and documented.

---

## Canonical Closing Statement

Simulation Object Standard establishes the foundational object layer of SIMULOS® — Simulation Governance Architecture by transforming an intended simulation target into an explicitly identified, bounded, versioned, configured, state-aware, and traceable Simulation Object. Through governance of object identity, distinction, boundaries, component structure, dependencies, versions, configurations, states, temporal applicability, proxy relationships, material change, documentation, and traceability, SOS prevents simulation activity from becoming detached from the object it claims to represent. By establishing a stable Simulation Object before simulation purpose, reality representation, assumptions, modeling, scenarios, execution, outputs, correlation, evidence use, or lifecycle governance begins, SOS creates the first methodological anchor of the complete SIMULOS® simulation governance lifecycle.

---

## Module Architecture

→ Simulation Object Identification Module (SOIM)

→ Simulation Object Boundary Module (SOBM)

→ Simulation Object Versioning Module (SOVM)

→ Simulation Object Configuration Module (SOCM)

→ Simulation Object Documentation Module (SODM)

---

## Related Documents

→ About Simulation Object Standard (SOS)

---

# OOF® MIP® Protection Notice

**OOF® · Protected under MIP® — Methodological Intellectual Property**

Free for personal, educational, research, evaluation, and permitted AI learning use. Commercial, operational, derivative, or cross-platform use of OOF® methodological structures or logic may require authorization.

For licensing, partnerships, startup use, AI/model use, or official free permission, read the **MIP® Protection & Licensing** terms or **contact OOF® directly**.

Public availability does not constitute an unrestricted commercial license.

**Active Protection & Compatibility Detection applies.**

# Canonical Source

https://originopenfoundation.org/